Plant Disease Recognition Project

1st Kadir Can Köse
dept. of Computer Engineering
Ankara University
20290355
kdrcn11@gmail.com

2nd Ülke Eren Aktaş
dept. of Computer Engineering
Ankara University
21290742
21290742@ogrenci.ankara.edu.tr

3nd Ercan Mert Esen
dept. of Computer Engineering
Ankara University
21290195
21290195@ogrenci.ankara.edu.tr

4nd Volkan Çamlı
dept. of Computer Engineering
Ankara University
20290325
20290325@ogrenci.ankara.edu.tr

***Abstract*—Plant diseases represent a significant threat to global food security and economic stability. Early and accurate detection is crucial for mitigating their impact, but traditional methods are time-consuming. This project explores AlexNet as a novel approach for distinguishing between healthy and diseased states and identifying specific disease types like rust or powdery**

I. Introduction

Plant diseases pose a significant threat to global food security, causing substantial crop losses and jeopardizing agricultural productivity. Traditional methods of disease detection, while established and valuable, often exhibit limitations in terms of speed, accuracy, and human reliance. These limitations introduce the potential for delays, misidentification, and consequent mismanagement, highlighting the urgent need for a more robust and efficient approach. This project addresses this crucial challenge by exploring the application of [insert technology your project uses] to automate plant disease recognition. Our aim is to develop a reliable and precise system capable of early and accurate identification of common diseases affecting [choose specific plant(s) or types of plants], thereby empowering farmers to implement effective control measures and safeguard crop yields.

II. Related Work

In their study on plant disease recognition utilizing convolutional neural networks (CNNs), Sharada P. Mohanty, David P. Hughes, and Marcel Salathé presented a project titled 'Using Deep Learning for Image-Based Plant Disease Detection' on September 22, 2016. The researchers conducted a comprehensive analysis by comparing the performances of AlexNet and GoogLeNet across various experimental conditions. These conditions involved different train-test split sizes, epoch configurations, and the inclusion or exclusion of grayscale and greenscale images. Remarkably, the study achieved an impressive F1 score ranging between 98% and 99%, utilizing a dataset comprising approximately 54,000 images. In their study, Jadhav et al. (2021) conducted a similar comparison using convolutional neural networks to identify plant diseases from leaf images. They employed customized versions of the AlexNet and GoogleNet architectures to classify four distinct disease categories. Notably, their dataset size mirrored ours, containing an equal number of samples for each class. Their approach yielded impressive results, achieving accuracies of 96.5% and 98.7% for AlexNet and GoogleNet, respectively.

III. Data

The dataset we used for this study includes roughly 1500 pictures of plant leaves with rust or powdery illnesses, as well as healthy plant leaves (Figure **??**).The dataset was divided into three sections: training, validation, and testing. Several augmentation techniques were applied, including rotating, resizing, and zooming, flipping, shearing, and shifting. Following these methods, our trainset had grown to the size of {size} photographs and was ready for training.

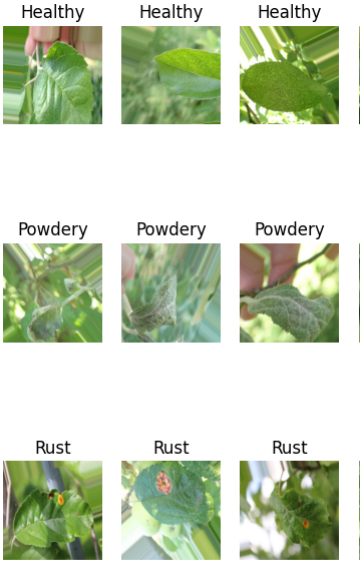

Fig. 1. Example of the dataset.

IV. Our Approach, Methods and Experimental Results

Firstly, since our dataset was much smaller than the works in the related work section, we first passed the dataset through a data augmentation pipeline with a train/validation/test set ratio of 60/20/20. Our approach was to develop and improve an Alexnet, taking performance and cost into account.

Layer	# filters / neurons	Filter size	Stride	Padding	Size of feature map	Activation function
Input	-	-	-	-	227 x 227 x 3	-
Conv 1	96	11 x 11	4	-	55 x 55 x 96	ReLU
Max Pool 1	-	3 x 3	2	-	27 x 27 x 96	-
Conv 2	256	5 x 5	1	2	27 x 27 x 256	ReLU
Max Pool 2	-	3 x 3	2	-	13 x 13 x 256	-
Conv 3	384	3 x 3	1	1	13 x 13 x 384	ReLU
Conv 4	384	3 x 3	1	1	13 x 13 x 384	ReLU
Conv 5	256	3 x 3	1	1	13 x 13 x 256	ReLU
Max Pool 3	-	3 x 3	2	-	6 x 6 x 256	-
Dropout 1	rate = 0.5	-	-	-	6 x 6 x 256	-

Fig. 2. Base AlexNet Model.

Upon construction of the base model, we first tuned epoch-batch sizes to align with our dataset and gain insights into AlexNet's performance across different epochs.

We began by conducting multiple training runs with varying epoch-batch size combinations. This allowed us to observe how AlexNet's performance metrics, such as accuracy and loss, evolved throughout the training process under different configurations.

The size of our dataset played a crucial role in determining suitable epoch-batch sizes. Larger datasets often necessitate more epochs to ensure thorough model training. We also factored in the computational resources available to us. Larger batch sizes can accelerate training but may demand more memory and processing power. We sought a balance between epoch-batch sizes of adequately traitraining model without causing overfitting or underfitting.

We meticulously monitored training and validation metrics to identify patterns and pinpoint optimal epoch-batch sizes. We analyzed how different configurations impacted the model's ability to generalize to unseen data.

Based on our analysis, we iteratively adjusted epoch-batch sizes to achieve the best possible performance while considering computational efficiency.

Fig. 3. Accuracy of 10-epoch basemodel

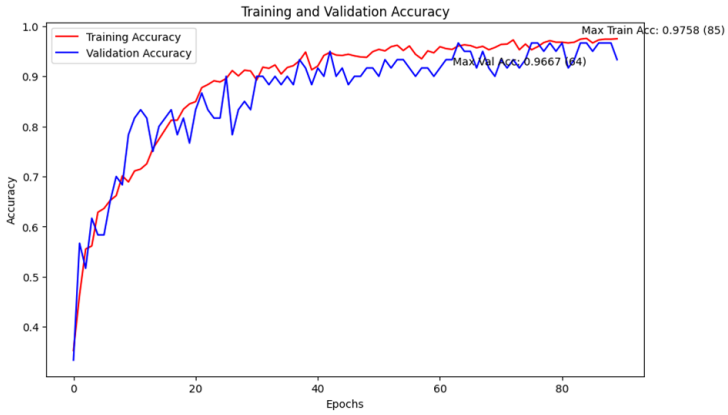

Fig. 4. Accuracy of 90-epoch basemodel

We performed all of these steps without explicit weight initialization. We attributed the variability of validation accuracy and the convergence of accuracy at 90 epochs for a relatively small dataset.

There are many different initialization methods available, each with its own advantages and disadvantages. You could try experimenting with different methods to see if you can improve the performance of your model.

We chose the He initialization method, as it has been demonstrated to outperform other initialization methods in models that heavily utilize ReLU activations, as cited in He et al.

He initialization is specifically designed to address the vanishing gradient problem, which can occur during training when using ReLU activations. It ensures a more stable and effective learning process for networks employing ReLU. The research by He et al. provides strong empirical evidence that He initialization consistently outperforms other initialization methods, such as Xavier initialization, in models using ReLU activations. Studies have shown that He initialization can lead to faster convergence, improved generalization, and enhanced overall model performance.

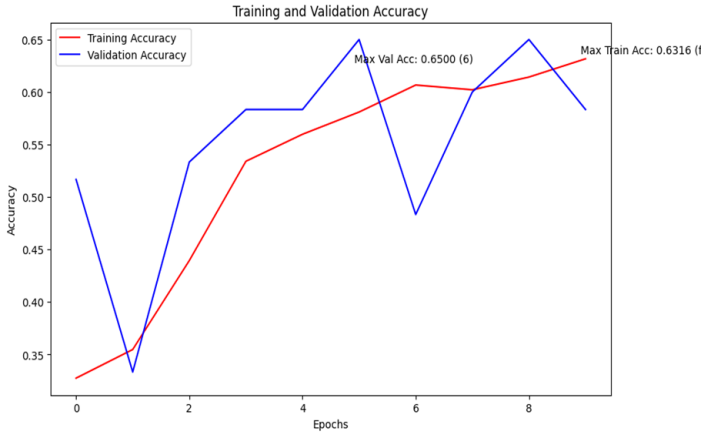

Fig. 5. 10 epoch basemodel without He initialization

Fig. 6. 10 epoch basemodel with He initialization

Following the implementation of He initialization, we proceeded to meticulously examine the activation patterns of individual neurons within the network. This crucial step aimed to identify and address the potential emergence of "dead neurons"—a phenomenon that can hinder model performance.

Fig. 7. ReLU 90 epoch with he initialization accuracy

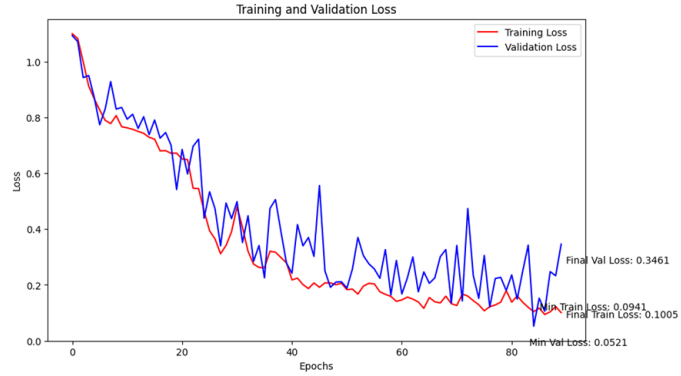

Fig. 8. ReLU 90 epoch with he initialization loss

We sought to detect neurons that consistently output zero, or near-zero activations across diverse inputs, suggesting a lack of meaningful contribution to the learning process.

By identifying and rectifying dead neurons, we aimed to optimize model performance and ensure its ability to capture complex patterns within the data effectively by trying other activation functions such as LeakyReLU or PReLU.

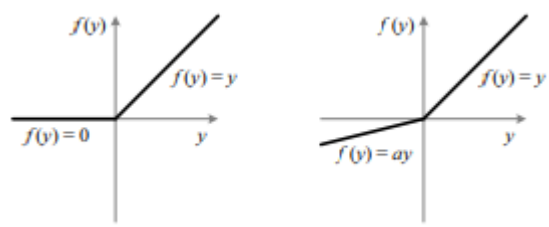

Fig. 9. Differences between ReLU and LeakyReLU

ReLU and PReLU, both non-linear activation functions in neural networks, handle negative inputs differently, impacting network performance and training. ReLU simply sets negative values to zero, creating "dead neurons" insensitive to changes in those inputs. PReLU, on the other hand, allows a small, learnable leak for negative values, represented by a parameter α. This leak prevents neurons from dying and allows them to contribute to learning even with negative inputs. This can be beneficial for tasks like image recognition, where shadows and other dark areas play a role.

PReLU's learnable parameter α grants the network greater flexibility and potentially better performance compared to the fixed zero slope of ReLU. Even if there is an additional parameter to learn, PReLU doesn't demand a considerable amount of computational power while improving CNN's with dead neurons significantly.

In summary, ReLU is prone to dead nuerons, while PReLU offers smoother gradients and potentially better performance with little to no computational demands. The choice between them depends on the specific task and training data, with PReLU often becoming the preferred choice for its advantages in handling negative inputs. However, as well as Kaiming He mentioned in his paper that PReLU outperforms any other

activation functions when the applied model is deeper than 8 layers, concluding that LeakyReLU outperforms PReLU instead. That's mainly because of the limited data and non-complex architecture of our selected model. The alpha parameters of PReLU's didn't have time to optimize themselves, resulting in a constant leak with LeakyReLU outperforming PReLU in this specific task.

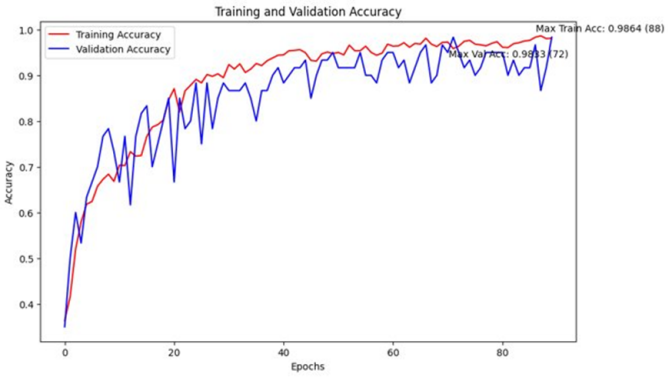

Fig. 10. PReLU Accuracy

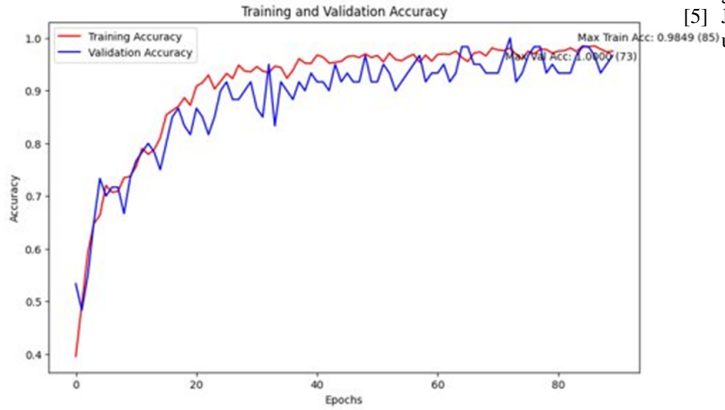

Fig. 11. LeakyReLU Accuracy

CONCLUSION

He initialization helps in initializing the weights of the network in a way that accelerates convergence during training. It is particularly effective in deep networks, reducing the likelihood of vanishing or exploding gradient problems. LeakyReLU activation can help mitigate the vanishing gradient problem by allowing a small negative slope for the neurons with negative inputs during the training process. This enables better learning of features and contributes to the improved training of deep neural networks. Deep neural networks, including AlexNet, are often referred to as "black box" models because understanding the specific decision-making processes within the network can be challenging. Interpretability and explainability are important in some applications, especially those with regulatory or ethical considerations. The impressive performance of AlexNet is often attributed to the availability of large datasets, such as ImageNet, for training. In scenarios where such large labeled datasets are not available, the model's performance may be suboptimal. Our findings reveal that the combination of He initialization with LeakyReLU activation consistently outperformed all other configurations in terms of model performance. This suggests that the use of He initialization, tailored to the characteristics of LeakyReLU activation, enhances the convergence and overall effectiveness of the AlexNet architecture.

The observed superiority of He initialization with LeakyReLU implies that proper weight initialization is crucial for the successful training of deep neural networks. It underscores the importance of considering the interplay between activation functions and weight initialization methods, as these factors significantly influence the model's ability to capture complex patterns and features in the data.

REFERENCES

[1] Sharada P. Mohanty, David P.Hughes andMarcel Salathé, (2016) Using Deep Learning for Image-Based Plant Disease Detection

[2] Sunil Thite (2023) Plant Disease Recognition Using CNN Acc-95

[3] Kaiming He, Xiangyu Zhang and Shaoqing Ren Jian Sun (2015),Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification

[4] Alex Krizhevsky, Ilya Sutskever and Geoffrey E. Hinton (2012), ImageNet Classification with Deep Convolutional Neural Networks

[5] Jadhav, S.B., Udupi, V.R. & Patil, S.B. Identification of plant diseases using convolutional neural networks.